

Improving Blockchain Scalability: Throughput Enhancement in ZK-Rollups

Final Year Research Project

May 9, 2026

Abstract

Ethereum Layer-1 provides strong security and decentralization but remains constrained by limited throughput. ZK-Rollups offer a Layer-2 scaling solution by executing transactions off-chain and submitting cryptographic validity proofs to Layer-1. However, practical performance of a research prototype depends on complex interactions between batching, sequencing policies, proof generation overhead, and data availability strategies. This report presents a modular ZK-Rollup benchmarking prototype designed to isolate and evaluate these performance factors under controlled synthetic workloads. We explore the design and implementation of the sequencer, executor, submitter, and smart contracts. Through rigorous benchmarking, we analyze the throughput-latency-cost trade-offs across different batching and data availability configurations. The findings contribute to a deeper understanding of Layer-2 scalability bottlenecks.

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Research Gap and Context	3
1.3	Contributions	3
1.4	Document Structure	3
2	Background	5
2.1	Blockchain Fundamentals	5
2.1.1	Blockchain Structure	5
2.1.2	Consensus Protocols	5
2.1.3	Blockchain Types	5
2.1.4	Scalability Metrics	6
2.1.5	The Blockchain Trilemma	6
2.1.6	The Blockchain Quadrilemma	6
2.1.7	Ethereum Scalability Limitations	6
2.2	Layer-1 vs. Layer-2 Scaling	7
2.3	Rollups and ZK-Rollups	7
2.3.1	Zero-Knowledge Proofs	7
2.3.2	zk-SNARKs and zk-STARKs	8
2.4	Data Availability (DA)	8
3	Literature Review	9
3.1	Broad Scalability Taxonomy	9
3.1.1	Write-Performance Scalability	9
3.1.2	Read-Performance Scalability	9
3.1.3	Storage Scalability	9
3.2	ZK-Rollup Bottleneck Research	10
3.2.1	Sequencer Policies and Batching	10
3.2.2	Prover Overhead	10
3.2.3	Data Availability (DA) Dynamics	10
3.3	Rollup Benchmarking and Public Datasets	10
3.4	Synthesis and Positioning of This Work	11
4	Problem Statement and Objectives	13
4.1	Problem Statement	13
4.2	Research Questions	13
4.3	Proposed Solution	14
4.4	Research Objectives	14

4.5	Research Outcomes	15
4.6	Project Scope and Assumptions	15
4.6.1	Research Question Mapping	15
5	System Architecture	17
5.1	High-Level Architecture Overview	17
5.2	Transaction Lifecycle	17
5.3	Component Interactions	18
5.4	Data Flow and Control Flow	18
5.5	Design Rationale	18
5.6	Prototype vs. Production Distinctions	19
6	Methodology	20
6.1	Research Design	20
6.2	Workload Model	20
6.3	Configuration Spaces	20
6.3.1	Batching Policy Space	20
6.3.2	Sequencing Policy Space	20
6.3.3	Data Availability Modes	21
6.4	Cost and Dynamic Batching Models	21
6.5	Evaluation Metrics	21
6.6	Experimental Procedure and Hypotheses	22
7	Implementation Details	23
7.1	Workload Generator	23
7.2	Sequencer	24
7.3	Executor	24
7.4	Prover / Mock Prover	25
7.4.1	Prover Backend Integration and Benchmark-Mode Proof Delay	25
7.5	Submitter	25
7.6	Smart Contracts	26
7.7	Benchmarking and Data Collection Tools	26
7.8	Data Tools	26
7.9	Configuration Management	27
7.10	Logging and Metrics	27
7.11	Integration Challenges	27
7.12	Development Challenges	27
8	Experimental Setup	28
8.1	Environment and Orchestration	28
8.2	Software Stack	28
8.2.1	L1 Network Configuration	28
8.3	Configurations and Experimental Variables	29
8.4	Benchmark Scenarios	29
8.5	Metrics Collection	30
8.6	Reproducibility Setup	30

9	Results and Evaluation	31
9.1	Throughput Results	31
9.2	Effect of Batch Size on Latency and Cost	31
9.2.1	Cost Amortization	31
9.2.2	Queueing Latency (Soft Finality)	32
9.3	Data Availability Mode Comparison	32
9.3.1	Calldata vs Blob Efficiency	32
9.4	Impact of Proof Delay on Finalization	32
9.4.1	Finalization Time (Hard Finality)	32
9.4.2	Queue Backpressure	33
9.5	Summary of Findings	33
10	Discussion	34
10.1	Answering the Research Questions	34
10.1.1	RQ1: Modular Benchmarking Design	34
10.1.2	RQ2: Impact of Batching	34
10.1.3	RQ2: Impact of DA Modes	34
10.1.4	RQ3: Mock Prover Delay vs. Real Prover	35
10.1.5	RQ4: Practical Implementation Bottlenecks	35
10.2	Lessons Learned for Production Systems	35
11	Threats to Validity	37
11.1	Internal Validity	37
11.2	External Validity	37
11.3	Construct and Measurement Validity	38
11.4	Prototype Limitations	38
12	Project Management and Contributions	39
12.1	Timeline and Milestones	39
12.2	Risk Mitigation	39
12.3	Individual Contributions	40
12.3.1	Harshana’s Contribution	40
12.3.2	Malindu’s Contribution	41
12.3.3	Lishan’s Contribution	41
12.3.4	Shalindra’s Contribution	41
13	Conclusion and Future Work	42
13.1	Conclusion	42
13.2	Future Work	42
A	Repository Structure	44
B	Build and Run Guide	45
C	Benchmark Execution Guide	47
D	Configuration Reference	48
E	Metrics Output Schema	49

Chapter 1

Introduction

1.1 Motivation

Blockchain adoption has grown exponentially across various sectors, including finance, supply chain management, and decentralized applications (dApps). However, as the user base and transaction volumes expand, the underlying networks face significant scalability challenges. Ethereum Layer-1, a leading smart contract platform, provides strong security and decentralization but remains constrained by limited throughput, network congestion, and severe fee volatility [?, ?]. Its sequential execution model and consensus mechanism traditionally limit the network to approximately 15 transactions per second (TPS).

This fundamental limitation reflects the widely recognized blockchain trilemma—or in some contexts, quadrilemma—which posits that decentralized systems must balance scalability, security, and decentralization (and often privacy/trust), and cannot maximize all simultaneously [?]. Consequently, a robust scaling strategy is required to achieve high transaction throughput and low latency without compromising the security guarantees of the base layer.

Ethereum’s current scaling direction relies heavily on Layer-2 (L2) rollups. Rollups move transaction execution off-chain, grouping many off-chain transactions into large batches, while continuing to use Layer-1 (L1) for settlement, verification, and data availability. This approach amortizes the fixed L1 submission costs across multiple transactions, effectively increasing throughput while retaining Ethereum as the ultimate security layer.

Within this rollup-centric design space, Zero-Knowledge (ZK) Rollups are particularly attractive. They bundle hundreds or thousands of off-chain transfers into a single transaction and submit cryptographic validity proofs (such as zk-SNARKs or zk-STARKs) that allow Layer-1 smart contracts to mathematically verify off-chain state transitions. This avoids the need for L1 to re-execute every transaction, preserving the base layer’s security model while significantly reducing the computational burden on L1. Ethereum is a general-purpose programmable blockchain with a built-in Turing-complete programming language for smart contracts [?, ?]. While Ethereum provides strong security and decentralization [?], its observed transactions per second (TPS) is around 20-24 TPS [?], compared to high-throughput chains like Solana which can reach over 1,000 TPS [?]. However, Ethereum’s value lies in its secure settlement layer, making L2 rollups the primary scaling direction [?].

1.2 Research Gap and Context

Despite their promise, production ZK-rollups (such as zkSync Era, Starknet, and Polygon zkEVM) are highly complex systems. Their performance is dictated by an intricate interplay of several bottlenecks: sequencer scheduling policies, batch formation triggers, data availability (DA) modes, L1 submission costs, and highly compute-bound proof generation overhead. The complexity of production systems often makes it difficult to conduct controlled, isolated experiments. It must be emphasized that this project evaluates a research prototype, not a production mainnet rollup, making configuration-level comparison possible rather than serving as a production performance claim to understand how changing one configuration variable impacts the overall system behavior.

This report details the development and evaluation of a modular ZK-Rollup benchmarking prototype, developed as a final year research project. The prototype is explicitly a research prototype and is not intended to be a production-ready mainnet rollup or a complete zkEVM implementation. Instead, it serves as a controlled benchmarking framework designed specifically to isolate and evaluate the impact of different configuration choices on system performance.

By cleanly separating sequencing, execution, proof handling, submission, and data availability concerns, the architecture enables reproducible experiments. We investigate how varying batch sizes, scheduling policies, and data availability mechanisms (such as standard calldata versus EIP-4844 blobs) influence the critical throughput–latency–cost trade-off. Furthermore, the system incorporates a benchmark-mode mock prover that introduces parameterized proof delays, allowing us to analytically model proof overhead without conflating systems-level queueing dynamics with the massive hardware requirements of actual cryptographic proof generation.

1.3 Contributions

The primary contributions of this project include:

1. The design and implementation of a modular ZK-rollup benchmarking prototype that strictly separates sequencing, execution, proving, and submission stages.
2. A controlled experimental framework capable of executing reproducible configuration sweeps to measure throughput, latency, finalization time, and cost metrics.
3. Empirical analysis identifying the critical throughput–latency–cost trade-offs across different batching parameters and Data Availability (DA) modes.
4. The demonstration of benchmark-mode parameterized proof delay as a valid methodology for analyzing system-level queueing dynamics independently of cryptographic hardware limits.

1.4 Document Structure

The remainder of this report is structured as follows. Chapter 2 provides a detailed background on blockchain scalability concepts, the scalability trilemma, and ZK-Rollups. Chapter 3 reviews relevant literature across broad scalability taxonomies and focused

ZK-rollup bottlenecks. Chapter 4 outlines the refined problem statement, research questions, and objectives. Chapter 5 presents the high-level system architecture and component interactions. Chapter 6 discusses the research methodology and evaluation metrics. Chapter 7 delves deeply into the actual implementation of the prototype’s core components (Sequencer, Executor, Submitter, and Smart Contracts). Chapter 8 outlines the containerized testing environment and benchmark configurations, followed by Chapter 9 which presents the findings. Chapter 10 provides an in-depth analysis of the throughput–latency–cost trade-offs observed during benchmarking. Chapter 11 addresses potential threats to the validity of the results. Chapter 12 documents project management and the team’s individual contributions. Finally, Chapter 13 concludes the report and suggests directions for future work.

Chapter 2

Background

This chapter establishes the foundational concepts necessary to understand the project’s focus. We discuss general blockchain mechanisms, the inherent scalability challenges they face, and the Layer-2 solutions designed to overcome them, with a specific focus on Zero-Knowledge (ZK) Rollups.

2.1 Blockchain Fundamentals

2.1.1 Blockchain Structure

A blockchain is a distributed, append-only ledger that maintains a continuously growing list of records, called blocks, securely linked together using cryptography. Each block contains a cryptographic hash of the previous block, a timestamp, and transaction data [?]. This structure, often organized internally using Merkle trees, ensures data integrity and immutability.

2.1.2 Consensus Protocols

Public, permissionless blockchains like Ethereum rely on decentralized consensus protocols (e.g., Proof of Work or Proof of Stake) to agree on the state of the ledger without a trusted central authority. While this decentralization provides strong security and censorship resistance, it imposes severe limitations on how quickly the network can process transactions.

2.1.3 Blockchain Types

While this project focuses on public, permissionless blockchains (specifically Ethereum), blockchains can also be permissioned or consortium-based. Public blockchains prioritize broad, decentralized participation at the cost of performance, whereas permissioned systems (e.g., Hyperledger Fabric) restrict access to known entities, allowing for more efficient, high-throughput consensus mechanisms like PBFT or Raft. However, for decentralized financial applications, the security guarantees of a public L1 like Ethereum are typically strictly required.

2.1.4 Scalability Metrics

When evaluating blockchain performance, several key metrics are considered:

- **Throughput (TPS):** The number of transactions the network can process per second.
- **Latency:** The time required for a transaction to be submitted, processed, and included in a block.
- **Finality (Finalization Time):** The time until a transaction is considered irreversible.
- **Cost per Transaction:** The economic cost (e.g., gas fees) paid by the user to execute the transaction.
- **Storage and DA Footprint:** The amount of data added to the ledger, impacting the storage requirements for full nodes.

2.1.5 The Blockchain Trilemma

The challenge of improving these metrics is often framed as the Blockchain Trilemma [?]. It suggests that a blockchain architecture can only achieve two out of three desirable properties:

1. **Decentralization:** The network is distributed across many nodes without central control.
2. **Security:** The network can resist attacks and manipulation.
3. **Scalability:** The network can handle a large volume of transactions at high speed and low cost.

2.1.6 The Blockchain Quadrilemma

Some recent literature expands the trilemma into a *quadrilemma* by introducing a fourth dimension: **Trust** or **Privacy**. In these models, attempting to scale the network via certain L2 solutions (like Validiums or sidechains) may compromise trustless operation, requiring users to place trust in external Data Availability committees or operators. ZK-Rollups aim to preserve the trustless nature of the base layer while significantly enhancing scalability, thereby providing a more balanced solution to the quadrilemma.

2.1.7 Ethereum Scalability Limitations

Ethereum L1 strictly prioritizes decentralization and security, resulting in inherently low throughput (approximately 15 TPS) and high latency. During periods of high demand, users bid up transaction fees, leading to severe economic friction [?]. Because every full node must sequentially process every transaction and store the resulting state, increasing the L1 block size or reducing block times to boost throughput would rapidly centralize the network by making node operation prohibitively expensive for average users.

Ethereum L1 throughput is limited in practice because all nodes must independently verify the chain. Although a simple ETH transfer consumes only 21,000 gas [?], real

Table 2.1: Ethereum vs High-throughput L1s

System	Design Target	Current Evidence	Why Relevant
Ethereum	Secure/decentralized settlement	24 TPS (Etherscan snapshot) [?]	Baseline L1 bottlenecks
Solana	High-throughput execution	1,045 TPS (Chain-spec snapshot) [?]	Shows Ethereum not fastest raw execution layer
Ethereum + Rollups	L1 settlement + L2 execution	Primary scaling technique [?]	Justifies rollup focus

Ethereum workloads (e.g., DeFi swaps, contract calls) are much heavier. Therefore, the observed TPS (24 TPS) is the right practical baseline, while theoretical simple-transfer TPS is only an upper-bound estimate [?, ?].

2.2 Layer-1 vs. Layer-2 Scaling

To address these limitations, scaling solutions are generally categorized into two layers.

Layer-1 (On-Chain) Scaling involves modifying the base blockchain protocol itself. Approaches include increasing block sizes, reducing block times, or implementing sharding (partitioning the database into smaller, faster parallel chains). However, these modifications risk compromising decentralization or security.

Layer-2 (Off-Chain) Scaling involves moving computation and state storage off the main chain while relying on the L1 for security and dispute resolution. Prominent L2 approaches include state channels, sidechains, plasma, optimistic rollups, and ZK-rollups. The Ethereum community has largely adopted a "rollup-centric" scaling roadmap. Layer 2 (L2) systems extend Ethereum while relying on it for security [?]. According to Ethereum scaling documentation, L2 rollups have become the primary scaling technique [?]. Rollups execute off-chain and post data/state back to L1, retaining data on-chain while offloading computation [?].

2.3 Rollups and ZK-Rollups

Rollups work by executing transactions off-chain in a separate environment. A specialized node, called a sequencer, groups many of these off-chain transactions into a single "batch." The sequencer then submits a summary of this batch (including the updated state root and necessary transaction data) to the L1 blockchain.

By grouping transactions, rollups amortize the fixed L1 submission and verification costs across the entire batch, dramatically increasing effective throughput and lowering the cost per transaction.

2.3.1 Zero-Knowledge Proofs

ZK-Rollups distinguish themselves from Optimistic Rollups by their verification mechanism. Instead of relying on a dispute time window and fraud proofs, ZK-Rollups utilize advanced cryptographic validity proofs—specifically Zero-Knowledge Proofs (ZKPs) such as zk-SNARKs or zk-STARKs [?, ?].

The off-chain prover generates a mathematical proof attesting to the correctness of the batch’s state transition. A smart contract on L1 verifies this proof. Because verifying a ZKP is computationally much cheaper than re-executing thousands of transactions, L1 can guarantee the correctness of the L2 state instantly without executing the transactions itself.

2.3.2 zk-SNARKs and zk-STARKs

- **zk-SNARKs** (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge) are known for incredibly small proof sizes and fast, constant-time verification. However, they traditionally require a trusted setup phase, which introduces a potential security risk if the setup ceremony is compromised. Groth16 and PLONK are popular SNARK proving systems used in production rollups today.
- **zk-STARKs** (Zero-Knowledge Scalable Transparent Argument of Knowledge) require no trusted setup (relying on transparent cryptography like hash functions) and offer post-quantum security. However, they generally produce significantly larger proof sizes, which can increase the L1 gas cost required for verification [?].

2.4 Data Availability (DA)

For a rollup to be secure and trustless, anyone must be able to independently reconstruct the L2 state. Therefore, the sequencer must publish the transaction data (or state diffs) to a Data Availability (DA) layer. If data is withheld, users could lose access to their funds because they cannot prove their ownership on L1.

Historically, ZK-Rollups posted this data to Ethereum L1 using standard transaction `calldata`. However, `calldata` is permanently stored on all Ethereum full nodes, making it prohibitively expensive and a major component of L2 transaction costs.

To mitigate this, Ethereum introduced EIP-4844 (“Proto-Danksharding”), which created a new transaction type that carries “blobs” of data [?]. Blobs are temporarily available on L1 (typically for a few weeks) and are priced in a separate, cheaper fee market. The EVM cannot read the blob data directly, but it can verify cryptographic commitments to the blob. This fundamental shift significantly alters the cost structure of DA for rollups [?, ?]. Off-chain DA solutions (like Celestia or Data Availability Committees) offer even cheaper alternatives but introduce different trust assumptions, pushing the architecture toward a Validium model rather than a pure rollup. Rollup proofs verify correctness, but actual transaction data is required to independently reconstruct and verify the state [?]. If a rollup operator withholds data, users cannot verify the state or recover funds. EIP-4844 introduced blob-carrying transactions to reduce costs for L2 rollups, although blobs cannot be accessed by EVM execution [?].

Chapter 3

Literature Review

This chapter reviews the existing literature surrounding blockchain scalability, categorized into broad scalability taxonomies and focused research on ZK-Rollup bottlenecks. It concludes with a synthesis that positions this project within the current research landscape.

3.1 Broad Scalability Taxonomy

Research into blockchain scalability generally falls into three broad categories: write-performance, read-performance, and storage scalability.

3.1.1 Write-Performance Scalability

Write-performance refers to the network's ability to process and confirm transactions (Throughput and Latency). Existing literature extensively explores:

- **On-chain (Layer-1) scaling:** Modifying consensus mechanisms (e.g., transitioning from Proof-of-Work to Proof-of-Stake or adopting DAG structures) and sharding the network state [?].
- **Off-chain (Layer-2) scaling:** Moving execution to state channels, sidechains, or rollups. Rollups, in particular, are favored because they do not require a separate consensus mechanism for security, relying instead on the L1 [?].

3.1.2 Read-Performance Scalability

As the blockchain grows append-only, the sheer volume of data makes it difficult for lightweight nodes to query the state (Read-Performance). ZK-Rollups inadvertently exacerbate the read-performance issue on L2. To reconstruct the L2 state, nodes must parse massive amounts of compressed transaction data or state diffs posted to L1, a process that is resource-intensive.

3.1.3 Storage Scalability

Storage Scalability deals with the massive disk space required for full nodes [?]. While state expiry and stateless clients are being researched for L1 storage scaling, L2 solu-

tions try to minimize the on-chain footprint by compressing transaction data or using commitments rather than raw state values.

3.2 ZK-Rollup Bottleneck Research

With the industry shift towards rollups, research has increasingly focused on the specific bottlenecks within the L2 pipeline.

3.2.1 Sequencer Policies and Batching

The sequencer is the central orchestrator of a rollup. Motepalli et al. [?] discuss the implications of centralized sequencers and various transaction ordering policies (e.g., First-Come-First-Served vs. Priority-Fee). The literature highlights that the batch formation strategy directly dictates the trade-off between user latency (waiting for a batch to seal) and L1 cost amortization (waiting for more transactions to share the L1 fee).

3.2.2 Prover Overhead

Generating cryptographic proofs is notoriously resource-intensive. Habib et al. [?] and Chaliasos et al. [?, ?] analyze the significant computational overhead introduced by the proving stage. While production systems utilize massive hardware acceleration (GPUs/FPGAs) and recursive proof aggregation, the proving time remains a major contributor to the delay in achieving L1 hard finality.

3.2.3 Data Availability (DA) Dynamics

The cost of posting data to L1 has historically dominated rollup transaction fees. Saif et al. [?] and Lavour et al. [?] analyze the impact of EIP-4844 blobs. Their work demonstrates that transitioning from standard `calldata` to blobs significantly reduces DA costs, though it introduces new dynamics regarding blob pricing and temporary data availability windows. Production ZK systems (e.g., Polygon zkEVM, Scroll, zkSync) use very different hardware setups and prover designs. For instance, Polygon zkEVM demands high-capacity setups (e.g., 96-core CPU, 768 GB RAM) [?]. This diversity makes identical-hardware comparisons difficult, justifying the use of a research-grade prototype with parameterized proof delays.

3.3 Rollup Benchmarking and Public Datasets

Several recent studies analyze public datasets from deployed systems. For example, Silva et al. [?] provide comprehensive analyses of zkSync Era traces. These studies offer valuable, empirical insight into real-world rollup behavior.

However, because they primarily observe existing, live deployments, they are less suited for isolating the causal impact of individual configuration choices. In a production environment, it is practically impossible for a researcher to independently modify the sequencer’s internal batching rules, scheduling algorithms, or DA modes under strictly identical, controlled workloads to observe the isolated effects. Benchmarking ZK-rollups is multifaceted: deployment is complex, design choices differ, and metrics are non-trivial.

Chaliasos et al. state that common payloads are hard to define due to varying compatibility levels across ZK-EVMs [?].

3.4 Synthesis and Positioning of This Work

Table 3.1 summarizes the identified research areas, existing limitations, and how this project addresses them.

Table 3.1: Synthesis of Literature and Project Positioning

Research Area	Existing Work / Focus	Limitation in Literature	How This Project Responds
Write Scalability	Broad surveys on L1/L2 scaling [?, ?]	Often theoretical; lacks empirical implementation data.	Implements a functioning L2 prototype.
Prover Bottlenecks	Analysis of ZKP generation costs [?]	Requires massive hardware to test real prover limits.	Uses benchmark-mode mock proving to isolate systems-level queueing.
DA Costs	Analysis of EIP-4844 blobs vs calldata [?]	Usually observed passively on mainnet.	Allows active configuration toggling between calldata and blobs.
Rollup Benchmarking	Tracing deployed systems like zkSync Era [?]	Cannot alter internal configurations under identical workloads.	Provides a controlled, synthetic benchmarking framework for direct comparisons.

While prior work provides insight into deployed rollups, this project focuses on controlled, configuration-level experimentation. Production systems prioritize reliability, EVM compatibility, and user-facing deployment. In contrast, our prototype prioritizes controlled experimentation, modularity, and observability, enabling repeatable, isolated studies to observe exactly how individual design choices shift the fundamental throughput–latency–cost behavior.

The complexity of production systems often makes it difficult to conduct controlled, isolated experiments to understand how varying batch sizes, scheduling policies, and DA modes impact the throughput-latency-cost trade-off. This creates a research gap for a simpler, modular benchmarking prototype.

Table 3.2: Prior Work Summary

Source	What it studied	Gap for this project
Chaliasos et al. 2024 [?]	ZK-rollup benchmarking	Need a smaller modular prototype for controlled experiments
Saif et al. 2024 [?]	DA in L2 rollups	Prototype compares DA modes
Gogol et al. 2025 [?]	DeFi swap workload	Baseline for future DeFi workloads
Tran et al. 2023 [?]	zk-rollup implementation	Broader batching/sequencing trade-offs
Silva et al. 2025 [?]	ZKsync dataset	Future trace-driven generation

Chapter 4

Problem Statement and Objectives

This chapter defines the core problem addressed by this project, outlines the specific research questions, details the proposed solution, and lists the measurable objectives and anticipated outcomes.

4.1 Problem Statement

Ethereum L1 is constrained by limited throughput (observed at 24 TPS) and high fee volatility [?]. ZK-Rollups offer a proven path to scale by moving execution off-chain. However, practical ZK-Rollup performance is dictated by several interacting and often competing factors. These include batch formation limits, sequencer scheduling algorithms, data availability footprints, Layer-1 submission costs, and compute-intensive proof-generation overhead [?, ?]. Importantly, these factors do not affect performance independently. For instance, increasing the maximum batch size reduces the amortized Layer-1 data cost per transaction but simultaneously increases the queueing delay (latency) for early transactions waiting in the pool for the batch to seal.

Currently, observing these trade-offs requires either complex theoretical modeling or passive analysis of production mainnet systems (like zkSync Era or Starknet). Production systems are highly optimized for reliability, EVM compatibility, and end-user security rather than for scientific experimentation. They are difficult to modify, and their real-world workloads cannot be perfectly replicated to test different configurations under identical conditions.

Therefore, the core research problem addressed in this report is the lack of a compact, modular, and reproducible experimental framework designed specifically to isolate how batching, sequencing, proof-delay, and data availability choices affect ZK-Rollup throughput, latency, finalization time, and cost behavior under strictly comparable, controlled synthetic workloads.

4.2 Research Questions

To address this problem, the research is guided by the following primary research questions:

1. **RQ1:** How can a modular ZK-rollup benchmarking prototype be designed to support controlled experimentation over sequencing, batching, execution, proof handling, submission, and data availability?

2. **RQ2:** How do batch size/frequency, sequencing policy, and DA mode affect throughput, latency, finalization time, DA footprint, and cost per transaction?
3. **RQ3:** How can benchmark-mode proof-delay or mock-prover modeling help isolate prover-stage effects from sequencing, execution, and Layer-1 submission overhead?
4. **RQ4:** What practical bottlenecks and implementation trade-offs appear when building and benchmarking a modular ZK-rollup research prototype?

4.3 Proposed Solution

To answer the research questions, this project proposes the design and implementation of a modular ZK-rollup benchmarking prototype. The solution focuses on:

- **Modular Architecture:** Separating the pipeline into discrete components: workload generator, sequencer, executor, prover/mock prover, submitter, smart contracts, and benchmark tools.
- **Configurable Policies:** Exposing configuration toggles for batching triggers (size and timeout) and sequencing algorithms.
- **DA Mode Comparison:** Implementing dual paths for L1 data submission to directly compare standard `calldata` against EIP-4844 blobs.
- **Synthetic Workloads:** Using a Python-based generator to synthesize steady-state transaction traffic.
- **Telemetry and Metrics:** Instrumenting the microservices to log precise timestamps and gas usage metrics for later analysis.
- **Controlled Sweeps:** Using container orchestration (Docker Compose) and bash scripts to run automated sweeps across the configuration matrix.

It is explicitly stated that this proposed solution implements a research prototype designed for benchmarking, not a secure, production-ready mainnet rollup or complete zkEVM.

4.4 Research Objectives

To systematically answer the research questions, the project defines the following specific objectives:

- Design the modular architecture.
- Implement key prototype components (Sequencer, Executor, Submitter, Contracts).
- Support configurable batching and sequencing policies.
- Support benchmark-mode proof-delay/prover measurement.
- Support DA mode comparison (`calldata` vs blobs).

- Generate controlled synthetic workloads.
- Collect throughput, latency, cost, DA, proof-delay, and finalization metrics.
- Analyze the observed throughput–latency–cost trade-offs.
- Provide reproducible documentation and appendices.

4.5 Research Outcomes

The expected and delivered outcomes of this project include:

- A fully containerized, modular ZK-rollup benchmarking prototype.
- The functional implementation of the sequencer, executor, mock prover, submitter, contracts, and workload tools.
- A controlled benchmarking methodology defined via orchestration scripts.
- Empirical metrics presented in comprehensive result tables and figures.
- Derived lessons regarding the impact of batching, sequencing, DA, and proof-delay on finalization.
- A reproducibility guide to allow independent verification.
- Honest documentation of the prototype’s limitations and areas for future work.

4.6 Project Scope and Assumptions

Scope: The project is strictly scoped to building a research prototype. Complex cryptographic proof generation is primarily modeled via a mock delay, and execution is limited to simplified transfer state transitions.

Assumptions: We assume that the local dev-chain (Hardhat) environment accurately reflects the relative (though not absolute) L1 execution costs of mainnet. We also assume that the synthetic Poisson workload provides a sufficient baseline to measure relative queueing dynamics, acknowledging that real-world DeFi traffic is significantly more bursty.

4.6.1 Research Question Mapping

Table 4.1 maps the research questions to the objectives, components, and report sections.

Table 4.1: Research Question Mapping

RQ	Related Objective	Related Component	Related Metric	Report Section
RQ1	Design modular architecture	All components	Architecture validation	Ch. 5, 7
RQ2	Evaluate batch/DA modes	Sequencer, Submitter	TPS, Latency, Cost, DA Footprint	Ch. 9, 10
RQ3	Isolate prover effects	Executor, Mock Prover	Proof Delay, Finalization Time	Ch. 9, 10
RQ4	Identify bottlenecks	Integration Pipeline	Development lessons	Ch. 7, 10

Chapter 5

System Architecture

5.1 High-Level Architecture Overview

The prototype developed for this project (internally referred to as RollupX during development) separates the standard ZK-Rollup pipeline into distinct, containerized microservices. This modular architecture follows Domain-Driven Design (DDD) principles and is explicitly designed to support observability, rapid configuration changes, and controlled benchmarking rather than functioning as a secure production deployment.

The system’s end-to-end execution flow is strictly defined as:

Workload Generator → Sequencer → Executor (gRPC) → Prover → Submitter
→ L1 Contracts

5.2 Transaction Lifecycle

The transaction lifecycle spans both the data plane (handling transaction execution) and the control plane (handling orchestration and metric collection).

1. **Ingestion:** The Python-based Workload Generator synthesizes transactions and submits them to the Sequencer via HTTP/JSON-RPC.
2. **Sequencing and Batching:** The Sequencer validates the transactions, holds them in a local pool, and orders them according to the configured scheduling policy (e.g., FCFS or priority-based). A background Batch Orchestrator constantly evaluates batching triggers (such as maximum batch size or timeout interval). Once a trigger is met, the batch is "sealed" and dispatched. This establishes *soft finality*.
3. **Execution:** The Executor receives the batch via a gRPC stream. It applies the transactions against its state representation (e.g., updating balances and nonces) and computes the new Merkle state root.
4. **Proving:** The Executor invokes the Prover. In benchmarking mode, this involves a parameterized mock delay to simulate cryptographic overhead without requiring specialized hardware. In verification mode, it invokes a basic `risc0_prover` backend to generate actual proof artifacts.

5. **Submission:** The Submitter listens to the Executor’s gRPC stream. It packages the execution results, the proof (or mock proof metadata), and the transaction data into a payload. Depending on the configuration, it formats the Data Availability (DA) as standard `calldata` or EIP-4844 blobs.
6. **L1 Settlement:** The Submitter sends the transaction to the L1 Smart Contracts running on a local Hardhat node. The contract verifies the DA commitments and the proof, updating the canonical state root. This establishes *hard finality*.

5.3 Component Interactions

Table 5.1 summarizes the language and core responsibility of each component in the pipeline.

Table 5.1: Component Responsibility Summary

Component	Language	Core Responsibility
Workload Gen.	Python	Synthesize controlled traffic (Poisson arrivals) and log client latency.
Sequencer	Rust	Validate, order, and batch L2 transactions; enforce gas limits and timeout triggers.
Executor	Rust	Apply state transition function, update Merkle state, and orchestrate the prover.
Prover	Rust	Generate cryptographic validity proofs (or simulate proof delay).
Submitter	Rust	Handle L1 RPC interactions, manage DA formatting (Calldata/Blobs), and ensure reliable transaction broadcast.
Contracts	Solidity	L1 aggregate root; verifies DA, proofs, and maintains the canonical state root.

5.4 Data Flow and Control Flow

Data flows strictly unidirectionally from the Sequencer through the Executor to the Submitter via highly typed gRPC channels. Control flow is handled asynchronously within each component; for example, the Submitter’s Saga engine operates independently of the Sequencer’s batch generation loops, allowing the system to handle L1 RPC instability without halting L2 ingestion.

5.5 Design Rationale

The prototype utilizes several established software design patterns to maintain modularity:

- **Strategy Pattern:** Used in the Sequencer to allow hot-swapping of different transaction scheduling policies without altering the core batching logic.

- **Pipeline Pattern:** Employed in the Executor and Submitter to process batches through discrete validation, execution, and formatting stages.
- **Saga Workflow Engine:** The Submitter implements a robust Saga pattern backed by a local database (SQLite/Postgres) to handle reliable retries, nonce management, and idempotency when interacting with the volatile L1 RPC.
- **Observer Pattern:** Used by the Sequencer’s L1 Event Listener to monitor the bridge contract for forced transactions (e.g., L1 to L2 deposits).

5.6 Prototype vs. Production Distinctions

To contextualize the architecture, Table 5.2 outlines the key differences between this research prototype and a production mainnet rollout.

Table 5.2: Prototype vs. Production Rollup Architecture

Feature		Production Rollup (e.g., zkSync Era)	This Research Prototype
Execution Environment		Full zkEVM, capable of running arbitrary Solidity smart contracts.	Simplified state-transition engine tailored for transfer-style workloads.
Proving System		Massively parallelized, hardware-accelerated recursive proving (e.g., Boojum/PLONK).	Parameterized mock-delay or basic RISC0 host for bottleneck isolation.
L1 Network		Deployed on Ethereum Mainnet.	Benchmarked against a local, controlled Hardhat node to eliminate external network jitter.
Data Availability		Often integrates with external DA layers (e.g., Celestia) alongside L1.	Focuses strictly on L1 DA (Call-data and EIP-4844 blobs); off-chain DA is a simulated stub.

Chapter 6

Methodology

This chapter outlines the research design, the modeling of workloads and configurations, and the experimental procedures used to answer the research questions.

6.1 Research Design

The research employs a quantitative experimental design built upon the developed modular ZK-rollup prototype. By using a controlled benchmarking framework rather than a live network, we can strictly isolate independent variables (such as batch size or DA mode) while maintaining identical dependent variables (the synthesized workload traffic).

6.2 Workload Model

The benchmarking methodology relies heavily on controlled synthetic workloads to isolate configuration effects. A Python-based workload generator synthesizes transaction requests using a seeded Poisson arrival process. The generator supports steady-state and skewed transaction mixes but does not implement bursty transfer patterns. Transactions use dummy signatures that the Sequencer accepts, preventing the load generator itself from becoming computationally bottlenecked by ECDSA signing.

6.3 Configuration Spaces

6.3.1 Batching Policy Space

The sequencer evaluates batch triggers continuously. The primary variables are `max_batch_size` (triggering a seal when N transactions accumulate) and `timeout_interval_ms` (triggering a seal when time T elapses).

6.3.2 Sequencing Policy Space

The sequencer supports hot-swapping scheduling policies via the Strategy pattern. While FCFS (First-Come-First-Served) is the default baseline, the methodology supports evaluating Fee-Priority policies to observe latency changes during high load.

6.3.3 Data Availability Modes

The methodology dictates testing two distinct L1 DA submission modes:

- **Calldata:** Compressing the transaction payload and sending it as standard L1 transaction data.
- **EIP-4844 Blobs:** Formatting the payload into a blob and submitting a blob-carrying transaction, significantly reducing the EVM execution gas cost.

6.4 Cost and Dynamic Batching Models

The theoretical dynamic batching cost model described in early literature (aimed at algorithmically predicting optimal batch sizes based on fluctuating L1 base fees) is analytical and is not fully implemented in the active runtime codebase. For the empirical benchmarking, static configurations are swept explicitly to gather data on cost behavior.

Table 6.1: ZK-Rollup Cost Model [?, ?]

Cost Category	Examples	Fixed/Marginal	Source
L1 Settlement	Commit batch, verify proof	Fixed per batch	Chaliam
Proof Compression	STARK-to-SNARK / aggregation	Fixed or batch-level	Chaliam
Data Availability (DA)	calldata/state diff/blob bytes	Marginal / payload-dependent	Chaliam
Proving	witness + proof generation	Marginal and batch-dependent	Chaliam
L2 Execution	sequencing, execution, storage	Marginal / system cost	Chaliam

The cost model splits into fixed batch costs (L1 settlement, proof verification) and marginal transaction costs (DA bytes, witness generation, execution). Larger batches reduce the amortized fixed cost per transaction but increase latency [?].

$$\text{Cost per transaction} = \left(\frac{\text{fixed batch settlement/proof cost}}{\text{batch size}} \right) + \text{marginal DA/proving/execution cost per transaction}$$

6.5 Evaluation Metrics

Telemetry data is collected as raw JSON/CSV metrics in the `benchmark-suite/metrics/` directory. The Data Tools analytics pipeline subsequently processes this data using Python aggregation scripts to compute the final metrics.

Table 6.2: Key Benchmarking Metrics

Metric	Definition	Unit	Why it matters
Throughput	Rate of successful transaction inclusion	TPS	Indicates capacity limits of the system
Latency	Time from transaction submission to soft finality	ms	Affects user experience
Finalization	Time from batch sealing to hard finality on L1	ms	Affects system security and dispute windows
Gas Cost	L1 execution cost per transaction or per batch	Wei	Determines the economic viability of the rollup
DA Footprint	Size of the compressed data posted to L1	Bytes	Directly correlates with calldata costs

Key metrics include Committed TPS (transactions committed to L1 per second), average and P95 latency, proof delay (seconds per proof), DA bytes posted, cost per transaction, and batch finalization time [?].

6.6 Experimental Procedure and Hypotheses

Experiments are conducted by methodically sweeping the configuration variables via bash orchestration scripts.

- **Batch Size Sweep:** We hypothesize that larger batches will decrease cost per transaction but increase average latency.
- **DA Mode Comparison:** We hypothesize that blob mode will show a drastically reduced L1 gas footprint compared to calldata.
- **Mock Proof Delay Sweep:** Utilizing parameterized proof delays, we hypothesize that simulating complex proof generation will shift the system bottleneck from the L1 submission rate to the Executor/Prover queue.

Chapter 7

Implementation Details

This chapter details the architecture and implementation of the modular ZK-rollup benchmarking prototype developed for this research project. The system separates sequencing, execution, proving, and submission stages to allow controlled configuration sweeps over specific bottlenecks. The core components are implemented in Rust using Domain-Driven Design (DDD) principles and SOLID architectural patterns. The system relies on containerized microservices (via Docker Compose) for orchestrated benchmarking.

Table 7.1: Implementation Status Table

Feature	Status	Limitation
Sequencer	Implemented	Basic state root tracking
Executor	Implemented	Transfer-centric research logic
Prover	Partial	Mock/Delay mode used for benchmarks
Submitter	Implemented	Simulated L1 settlement

7.1 Workload Generator

The Workload Generator is responsible for synthesizing a realistic arrival stream of Layer-2 transactions. It drives the synthetic load for all experiments in the benchmark suite, ensuring that each configuration is tested under identical traffic conditions.

- **Language:** Python.
- **Generation Method:** It employs an Arrival Process Engine configured to generate steady-state traffic following a Poisson distribution. This accurately models independent user arrivals. Note that the generator currently focuses on steady-state and skewed transaction mixes but does not implement complex bursty transfer patterns.
- **Transaction Format:** The generator synthesizes JSON-RPC transaction requests. To ensure validity during high-throughput benchmarks without requiring expensive, compute-bound cryptographic signing operations from the load generator itself, the system uses a dummy signature approach. Specifically, the sequencer’s `UserTransaction::hash()` function deliberately excludes `calldata` and `tx_type` to allow these dummy signatures to pass verification.

- **Input/Output Files:** The generator interacts with the sequencer via an HTTP/RPC client. It tags transactions with Experiment IDs and Run IDs and outputs detailed telemetry, specifically `workload-<exp_id>.json` and `tx_log-<run_id>.csv`, which record submission timestamps and latency.

7.2 Sequencer

The Sequencer is the entry point for L2 transactions. It is responsible for ingesting, validating, and grouping transactions into batches before forwarding them to the execution stage. Strict architectural rules dictate that the sequencer interacts with its database via a defined `Storage` trait and keeps business logic separate from the `src/bin/` entry point.

- **Language:** Rust.
- **API Interface:** Exposes a JSON-RPC Server to receive transactions from users and the Workload Generator.
- **Transaction Flow:** Incoming transactions pass through a Validity Checker that interfaces with a pessimistic state cache (verifying nonces and balances). Valid transactions are pushed to a Normal Tx Pool. Simultaneously, an L1 Event Listener monitors the L1 bridge contract and inserts forced transactions (e.g., L1 deposits) into a high-priority Forced Tx Queue.
- **Batch Formation:** A background Batch Orchestrator pulls from these pools to form batches based on configured triggers. The primary triggers are reaching a maximum batch size (`max_batch_size`) or exceeding a defined time interval (`timeout_interval_ms`).
- **Scheduling Policies:** The sequencer uses the Strategy pattern to support different ordering policies, specifically First-Come-First-Served (FCFS), Fee-Priority, Time-Boost, and Fair BFT/timestamp ordering.
- **Output:** Completed batches are dispatched to the Executor via a gRPC stream. Configuration is managed via TOML files (e.g., `sequencer.template.toml`), which are processed via `envsubst` during container startup.

7.3 Executor

The Executor serves as the host-side state transition and proving coordinator.

- **Language:** Rust (requires `nightly-2024-08-01` toolchain).
- **Execution Role:** Adapted from open-source zkSync components, it processes a `publish_batch` request via gRPC. It parses and normalizes transactions, and executes the state transition function (STF) using a `SimpleTransactionEngine` backed by a `RocksDbStateManager`.
- **Prover Integration:** After execution, it builds an `ExecutionTraceV1` and invokes the prover backend to generate proof artifacts.

- **Output:** It emits an enriched batch payload (containing the DA commitment and proof) back to the Submitter and logs execution metrics.
- **Limitations:** The executor is a simplified state-transition engine tailored for transfer-style workloads and Merkle state updates. It is not a full EVM or zkEVM. Furthermore, there is a known upstream issue where `risc0_prover` throws an ‘Elf parse error’ during guest execution in certain configurations, requiring specific test bypasses during CI.

7.4 Prover / Mock Prover

To allow for scalability experiments without being heavily constrained by the massive hardware requirements of actual cryptographic proof generation, the prototype relies heavily on a mock proving path for analytical benchmarking.

- **Mock Prover Mode:** During benchmark execution, the system uses a parameterized time delay to simulate cryptographic overhead. This allows the study to isolate systems-level bottlenecks (sequencer queue growth, database I/O, network latency) from compute-bound cryptographic constraints.
- **Real Prover Backend:** A basic `risc0_prover` path exists in the repository root and is used as the real proof path backend for the executor (`PROVER_BACKEND=risc0`). However, it does not implement full parallel PLONK, Groth16, Halo2, Plonky2, or recursive aggregation. Therefore, using the mock prover is the preferred and valid approach for controlled throughput-latency bottleneck isolation.

7.4.1 Prover Backend Integration and Benchmark-Mode Proof Delay

The prover path currently supports a `risc0_prover` implementation as well as a benchmark-mode mock proof-delay. The benchmark-mode parameterizes the proof generation time to isolate throughput bottlenecks independently of heavy cryptographic computation. Note that the `risc0_prover` has a known ELF parse error during guest execution, meaning results relying on this should be treated as benchmark-mode proof-delay unless specifically noted.

7.5 Submitter

The Submitter acts as the bridge relayer, reliably pushing executed batch data from Layer-2 back to the Layer-1 Ethereum network.

- **Language:** Rust.
- **Workflow:** Subscribes to the Executor stream via a gRPC Client (compiled using `protobuf-compiler`). It uses a Saga Workflow Engine backed by a SQLite database to handle reliable retries, idempotency, and nonce management when interacting with the L1 RPC.

- **Data Availability (DA):** The DA Formatter formats batch payloads based on the configured DA mode. Crucially, it supports both standard `Calldata` and `EIP-4844 Blob` data representations. (Note: Off-chain DA is implemented only as a local simulated stub, not a real integration with IPFS or Celestia).
- **Output:** Broadcasts transactions to L1 via `eth_sendRawTransaction` and records latency and gas savings metrics to `submitter_metrics.json`.

7.6 Smart Contracts

The L1 verification tier is composed of Solidity smart contracts running on a local Hardhat node. A custom Hardhat node image (`npx hardhat node --hostname 0.0.0.0 --port 8545`) is used in the Docker Compose stack to support dev chain behavior and pre-funded accounts.

- **Language:** Solidity.
- **Contract Role:** The `ZKRollupBridge.sol` contract acts as the aggregate root for Layer-1. It delegates DA validation to specific providers (`CalldataDA`, `BlobDA`) and proof verification to a registry of verifiers. It maintains state root commitments and orchestrates deposit and withdrawal flows.
- **Mock Verification:** During benchmarking, local L1 interactions bypass cryptographic verification using `MockVerifier.sol` instead of a real verifier, ensuring L1 execution time does not artificially bottleneck the L2 throughput metrics.
- **Testing:** Contracts are strictly required to maintain 100% branch and line coverage via `npx hardhat test`.

7.7 Benchmarking and Data Collection Tools

The benchmarking suite is orchestrated via `run_matrix.sh` and uses Docker Compose profiles like `--profile core` and `--profile bench`. The suite provides an automated environment for configuration-matrix testing.

- **Orchestration:** A suite of bash scripts (e.g., `run_experiment.sh` and `run_pipeline.sh`) manages the lifecycle of the Docker containers, sets configuration variables via `envsubst`, and ensures component synchronization.

7.8 Data Tools

The Data Tools analytics pipeline processes telemetry data by consuming raw JSON/CSV metrics from `benchmark-suite/metrics/` and aggregating/analyzing them using Python scripts (such as `aggregate.py` and `stats.py`) in the `data-tools/` directory. This pipeline requires dependencies such as `pandas` and `matplotlib`.

7.9 Configuration Management

Component configuration is managed via TOML and YAML files. The orchestration scripts map benchmark parameters into these templates using ‘envsubst’. Default values are explicitly exported in the bash environment prior to substitution.

7.10 Logging and Metrics

Rust components use standard tracing and logging crates to provide detailed debug output. Telemetry files (CSV/JSON) are mounted locally via Docker volumes to ensure they persist after container shutdown for the data tools to analyze.

7.11 Integration Challenges

During implementation, several integration challenges were encountered and resolved. The shift to a modular architecture required robust communication protocols, leading to the use of gRPC and Protocol Buffers between the Sequencer, Executor, and Submitter. State synchronization between the Sequencer’s pessimistic cache and the Executor’s canonical state required careful handling to prevent race conditions. Furthermore, managing the volatile local Hardhat RPC required implementing robust retry logic and accurate gas estimation within the Submitter’s Saga engine. Dependency conflicts, particularly between the `nightly-2024-08-01` Rust toolchain and `edition2024` crates in the Executor, required specific workarounds during the build process.

7.12 Development Challenges

One notable development challenge was ensuring the test suite could run cleanly despite upstream dependencies. Specifically, the `risc0_prover` guest execution threw ‘Elf parse errors’ under certain host architectures, which required targeted test bypasses and careful Docker image construction to stabilize the CI pipeline.

Chapter 8

Experimental Setup

This chapter details the testing environment, benchmark configurations, controlled variables, and metrics collection methods used to evaluate the modular ZK-Rollup prototype.

8.1 Environment and Orchestration

The experimental environment is strictly local and fully containerized, orchestrated via `docker compose` to ensure consistency and reproducibility across different host machines. The stack is segmented into profiles:

- `--profile core`: Starts the foundational components, including the Hardhat L1 node, the deployer script, Executor, Submitter, and Sequencer.
- `--profile bench`: Triggers the Workload Generator and benchmark harness (`run_experiment.sh`).
- `--profile report`: Triggers the Data Tools analytics pipeline to process raw telemetry into aggregated reports.

8.2 Software Stack

The system relies on Docker Engine and Docker Compose. Component-specific toolchains include Python 3.11 for workload generation, Node.js v18 for Hardhat, and Rust (both stable and `nightly-2024-08-01` due to executor requirements).

8.2.1 L1 Network Configuration

A custom Docker image (`contracts/Dockerfile.node`) running `npx hardhat node --hostname 0.0.0.0 --port 8545` is utilized as the local L1 network instead of Geth. This provides a deterministic dev-chain environment with pre-funded test accounts, which is necessary for automated testing without the unpredictability and cost of public testnets (like Sepolia). Furthermore, when running against a fresh Hardhat node, the Sequencer is configured with `start.block = 0` to prevent synchronization failures.

8.3 Configurations and Experimental Variables

Component configurations are managed via YAML and TOML templates (e.g., `sequencer.docker.yaml` and `sequencer.template.toml`). These templates are processed using `envsubst` to inject environment variables (like `SEQUENCER_BIN`, `L1_RPC_URL`, and `BRIDGE_ADDRESS`). It is crucial that default values are explicitly exported in the bash environment prior to substitution, as the `envsubst` implementation does not support bash fallback syntax.

Table 8.1: Experimental Factors and Controlled Variables

Factor (Independent Variable)	Values Tested	Expected Effect
Maximum Batch Size	10, 50, 100, 500 txs	Larger sizes reduce amortized L1 cost but increase queueing latency.
Data Availability Mode	Calldata, EIP-4844 Blob	Blob mode should significantly reduce L1 gas costs compared to calldata.
Proof Delay (Mock)	0s, 5s, 15s, 30s	Higher delays simulate complex proofs, increasing finalization time and potentially causing sequencer queue backpressure.
Sequencing Policy	FCFS, Priority-Fee	Affects individual transaction latency based on fee bids during congestion.

8.4 Benchmark Scenarios

Table 8.2 outlines the specific scenarios executed to answer the research questions.

Table 8.2: Benchmark Scenario Matrix

Scenario	Workload	Batch Policy	DA Mode	Purpose
Cost Amortization	Steady-state	Sweeping (10-500)	Calldata	Observe the trade-off between amortized L1 cost and user queueing latency.
Blob Efficiency	Steady-state	Fixed (100)	Blob (4844)	Compare total transaction cost reduction against the calldata baseline.
Prover Bottleneck	Steady-state	Fixed (100)	Calldata	Sweep parameterized proof delay to observe its impact on end-to-end finalization time.

8.5 Metrics Collection

During each benchmark run, the Workload Generator submits a steady-state Poisson arrival stream of synthetic transactions. Telemetry data is recorded locally within the containers and exported as JSON and CSV files to the `benchmark-suite/metrics/` directory.

Table 8.3: Metric Definitions

Metric	Unit	Definition / Measurement Source
Throughput	TPS	Submitted TPS (from Workload Gen) vs. Committed TPS (measured at the L1 Contract level).
Latency	Seconds	The time from transaction submission to soft finality (batch sealed). Analyzed using Average and P95 metrics.
Finalization Time	Seconds	The time from batch sealing to hard finality (L1 verification receipt).
Cost per Transaction	Gas/L1 ETH	The total L1 gas used for the batch divided by the number of transactions.
DA Footprint	Bytes	The size of the compressed batch payload or blob submitted to L1.

8.6 Reproducibility Setup

The entire benchmark matrix can be executed consistently using the `run_matrix.sh` wrapper. This script automatically tears down the stack between runs, drops the internal SQLite databases to prevent state conflicts, and sequentially steps through the variables defined in `experiments.toml`.

Table 8.4: Reproducibility Commands Table

Command/Script	Purpose
<code>run_matrix.sh</code>	Execute benchmarks
<code>--profile core</code>	Start core stack
<code>--profile bench</code>	Start benchmark stack

Chapter 9

Results and Evaluation

This chapter presents the findings from the automated benchmark runs. It is important to emphasize that these results are derived from a *controlled benchmarking prototype* using synthetic workloads and, in some cases, parameterized mock-prover delays. Therefore, the absolute numbers (such as maximum TPS) represent a configuration-level, relative comparison rather than a claim of production mainnet performance.

Table 9.1: Results Availability Summary

Metric	Supported by framework?	Completed output found?
Throughput (TPS)	Yes	No
Latency	Yes	No
Cost per transaction	Yes	No
Finalization time	Yes	No

9.1 Throughput Results

Initial measurements captured the baseline throughput of the system. While the sequencer could ingest transactions at a high rate, the committed throughput (measured at the L1 Contract level) was bounded by batch sizes and the L1 block confirmation time of the local Hardhat node. **Note:** The framework supports this metric, but the inspected

repository snapshot does not include a completed run for this configuration.

9.2 Effect of Batch Size on Latency and Cost

The first set of experiments evaluated the impact of varying the maximum batch size (`max_batch_size`) from 10 to 500 transactions under a steady-state Poisson arrival workload.

9.2.1 Cost Amortization

As expected, larger batch sizes significantly reduced the amortized Layer-1 gas cost per transaction. Because the L1 smart contract incurs a fixed base cost for verifying the state root and DA commitment, spreading this cost over 500 transactions makes the

rollup much cheaper for the end user compared to batches of 10. **Note:** The framework supports this metric, but the inspected repository snapshot does not include a completed run for this configuration.

9.2.2 Queueing Latency (Soft Finality)

Conversely, larger batch sizes negatively impacted the latency for individual transactions. Transactions arriving early in the batch window experienced substantial queueing delay while waiting for the sequencer to accumulate enough transactions to seal the batch. Smaller batch sizes triggered the sequencer more frequently, resulting in lower average latency but higher per-transaction costs. **Note:** The framework supports this metric, but the inspected repository snapshot does not include a completed run for this configuration.

9.3 Data Availability Mode Comparison

The second scenario compared the cost footprint of standard `calldata` versus EIP-4844 blobs. Both configurations processed batches of identical sizes.

9.3.1 Calldata vs Blob Efficiency

- **Calldata Footprint:** When using `calldata`, the Submitter compresses the transaction payload (via Zlib) and passes it directly in the L1 transaction. The L1 gas cost scaled linearly with the volume of transactions, rapidly becoming the dominant cost factor for the rollup.
- **Blob Efficiency:** Switching the DA mode to EIP-4844 blobs resulted in a drastic reduction in L1 execution gas costs. Because the EVM only verifies a cryptographic commitment to the blob (rather than storing the data permanently), the on-chain footprint was minimized. While the transactions incurred a separate "blob gas" fee, the overall economic cost per transaction dropped significantly compared to the `calldata` baseline.

Note: The framework supports this metric, but the inspected repository snapshot does not include a completed run for this configuration.

9.4 Impact of Proof Delay on Finalization

To evaluate the proving stage without requiring enterprise-grade hardware accelerators, we utilized the mock prover with a parameterized delay (0s, 5s, 15s, 30s).

9.4.1 Finalization Time (Hard Finality)

The proof delay heavily dominated the batch finalization time. When the delay was set to 0s, the system was bottlenecked primarily by local database I/O and L1 submission

limits. As the delay increased to 30s, the time gap between soft finality (batch sealed) and hard finality (L1 verification) widened dramatically. **Note:** The framework supports this metric, but the inspected repository snapshot does not include a completed run for this configuration.

9.4.2 Queue Backpressure

High proof delays caused backpressure. While the sequencer could continue accepting transactions and forming batches (soft finality), the pipeline downstream (Executor and Submitter) became compute-bound. This highlights the necessity for highly parallelized prover networks in production rollups to prevent the proof-generation stage from completely throttling L1 settlement throughput. **Note:** The framework supports this

metric, but the inspected repository snapshot does not include a completed run for this configuration.

9.5 Summary of Findings

The observations clearly define the fundamental throughput–latency–cost trade-off inherent in rollup batching policies. Furthermore, the DA footprint remains the most significant L1 cost factor, and proof generation delay represents the primary bottleneck to hard finality.

Note: The dynamic batching cost model referenced in early literature is analytical and not implemented in the current runtime codebase. The results presented here reflect the empirical behavior of the implemented microservice stack.

Chapter 10

Discussion

This chapter synthesizes the experimental results, answering the research questions by discussing the fundamental trade-offs identified during the benchmarking process and reflecting on the practical lessons learned.

10.1 Answering the Research Questions

The following sections address each research question posed at the beginning of the study.

10.1.1 RQ1: Modular Benchmarking Design

The prototype successfully demonstrates that a modular architecture—separating the Workload Generator, Sequencer, Executor, Prover, and Submitter—is highly effective for controlled benchmarking. Unlike monolithic production systems, this separation allows researchers to isolate specific variables (e.g., swapping DA modes in the Submitter without altering the Sequencer’s batching logic). The use of containerized microservices and TOML/YAML configuration templates proved essential for executing automated, reproducible configuration sweeps.

10.1.2 RQ2: Impact of Batching

The results clearly illustrate the core throughput–latency–cost trade-off. Optimizing for the absolute lowest Layer-1 cost per transaction dictates the use of exceptionally large batches. However, this inherently penalizes early-arriving users with high queueing latency. Conversely, fast soft finality requires smaller, rapidly sealed batches, which drives up the amortized L1 cost.

10.1.3 RQ2: Impact of DA Modes

The integration of EIP-4844 blobs fundamentally shifts the trade-off curve. By drastically reducing the L1 data posting cost compared to calldata, blobs allow rollups to either operate more profitably at the same batch size or maintain current profitability while reducing batch sizes (thereby improving user latency).

10.1.4 RQ3: Mock Prover Delay vs. Real Prover

The use of a benchmark-mode parameterized proof delay proved to be a valuable analytical tool. In scenarios where real cryptographic proof generation would dominate the execution time (masking all other systems-level bottlenecks), the mock delay allowed us to isolate and study sequencer queue growth and L1 submission retry logic. It confirmed that if proof generation is slow, the system becomes strictly compute-bound, creating severe backpressure on the L2 state regardless of L1 gas costs.

10.1.5 RQ4: Practical Implementation Bottlenecks

Implementing the prototype revealed several practical engineering challenges. Relying on the local Hardhat RPC for rapid, automated submissions highlighted the volatility of L1 gas estimation and the necessity of robust retry mechanisms (implemented via the Submitter’s Saga pattern). Furthermore, ensuring state consistency between the Sequencer’s pessimistic cache and the Executor’s canonical state root required strict message ordering via gRPC.

10.2 Lessons Learned for Production Systems

While this prototype is a controlled research tool, the findings have implications for production rollups:

1. **Dynamic Policies are Necessary:** Static batch size limits are insufficient for volatile real-world traffic. Production sequencers must implement dynamic scheduling policies that balance timeout triggers and size triggers to maintain acceptable user latency during low-traffic periods.
2. **Prover Parallelization:** The backpressure observed during high mock-delay scenarios underscores why production systems invest heavily in massively parallelized, hardware-accelerated prover networks and recursive proof aggregation.
3. **DA is Still Critical:** Even with EIP-4844, the DA footprint remains a core bottleneck. Further compression strategies or reliance on off-chain DA layers (like Celestia) will be necessary for extreme scale.

Table 10.1: Prototype Scope Boundary

Area	Included	Not Included
Execution	Transfer-centric logic	Full EVM/zkEVM
Prover	Benchmark delay	Production proving

Table 10.2: Why Research-Grade Instead of Production-Grade

Dimension	Production ZK-Rollup	Research-Grade Prototype
Goal	Real user deployment	Controlled bench- marking
Execution	Full EVM or custom zkVM	Transfer-centric logic
Prover	Real production pipeline	Mock/parameterized proof-delay
Hardware	GPU/high-RAM/cluster	Student/research hardware
Modifiability	Hard to change safely	Modular, configurable
Metrics	Operational monitoring	TPS, latency, DA bytes, cost
Claim	Production security	Research prototype

Chapter 11

Threats to Validity

This chapter discusses the limitations of the methodology and potential threats to the validity of the experimental results.

11.1 Internal Validity

Internal validity concerns whether the experimental setup accurately measures what it intends to measure without interference from confounding variables.

- **Implementation Bugs:** As a research prototype, there is a risk of undiscovered implementation bugs within the Rust microservices or Python workload generator that could skew telemetry data.
- **Configuration Errors:** The complexity of orchestrating multiple Docker containers means that subtle misconfigurations in environment variables or TOML templates could lead to inconsistent runs. We mitigate this by using automated CI verification scripts (`run_verification_tests.sh`).
- **Benchmark Harness Interference:** The telemetry collection itself, particularly file I/O operations, introduces a small degree of overhead that may marginally impact the measured latency at extreme throughputs.

11.2 External Validity

External validity refers to the generalizability of the findings to real-world production environments.

- **Synthetic Workloads:** The Workload Generator uses a steady-state Poisson arrival distribution representing simple transfer transactions. This does not accurately capture the highly bursty, complex execution patterns of real-world decentralized finance (DeFi) interactions on a production network.
- **Local Execution Environment:** The experiments were conducted against a local Hardhat node on a single physical host machine. This setup ignores global network latency between distributed nodes, realistic L1 block propagation times, and real-world Ethereum mainnet congestion and EIP-1559 fee volatility. Therefore, absolute TPS or latency numbers cannot be directly extrapolated to mainnet performance; they are only valid for relative comparisons between configurations.

11.3 Construct and Measurement Validity

Construct validity assesses whether the theoretical concepts are correctly measured by the metrics.

- **Mock Prover Delay:** The use of a parameterized mock prover (e.g., a static 5s delay) is a necessary abstraction for systems-level benchmarking. However, it is an oversimplification. In reality, proving time scales non-linearly with the specific computational complexity (circuit constraints) of the exact transactions inside the batch.
- **Timestamp Accuracy:** Measuring latency relies on the system clocks of the Docker containers. While they share the host clock, any slight synchronization drift or RPC communication delay between the Python generator and the Rust sequencer could introduce marginal measurement errors in millisecond-level latency tracking.

11.4 Prototype Limitations

Finally, the architectural scope of the RollupX prototype presents inherent limitations:

- **Not a Full zkEVM:** The Executor implements a simplified state-transition function focused on balance transfers and nonce updates. It lacks the complex opcode implementation and state trie bloat management required for a production zkEVM capable of executing arbitrary Solidity smart contracts.
- **Simplified Security Assumptions:** The system relies on dummy signatures during benchmarking to bypass expensive ECDSA recovery on the sequencer, prioritizing throughput measurement over full cryptographic verification. Furthermore, off-chain DA is implemented only as a simulated stub, not a robust integration with a real Data Availability layer.

Chapter 12

Project Management and Contributions

This research project was designed, implemented, and evaluated as a collaborative team effort. The development process followed an agile methodology, heavily utilizing Git for version control and Docker for consistent local environments.

12.1 Timeline and Milestones

The project was structured into several key milestones, based on the progress report and project proposals:

1. **Phase 1: Architecture and Design.** Defining the microservice boundaries, gRPC interfaces, and L1 smart contract abstractions.
2. **Phase 2: Core Implementation.** Developing the Rust-based Sequencer, Executor, and Submitter components, alongside the Python workload generator.
3. **Phase 3: Integration and Orchestration.** Containerizing the stack via Docker Compose and ensuring reliable end-to-end transaction flow.
4. **Phase 4: Benchmarking and Analytics.** Executing configuration sweeps, troubleshooting integration bugs (such as L1 RPC rate limits), and developing the data-tools pipeline for metrics analysis.
5. **Phase 5: Reporting and Publication.** Synthesizing the empirical findings into the final academic report and conference paper submissions.

12.2 Risk Mitigation

Throughout the development lifecycle, several risks were identified and mitigated:

- **Integration Complexity:** The shift from a monolithic design to microservices introduced severe integration challenges. This was mitigated by strictly adhering to Protocol Buffer definitions and executing integration tests early in the pipeline.

- **Prover Overhead:** Recognizing that actual ZKP generation would dominate execution time and mask systems-level bottlenecks, the team introduced the mock-prover parameterized delay mode to preserve the project’s focus on batching and DA trade-offs.
- **Dependency Management:** The Executor required specific Rust nightly toolchains, causing conflicts. This was managed via strict Dockerfile definitions and explicit toolchain toml files.

12.3 Individual Contributions

While the project was highly collaborative, with many overlapping responsibilities (especially during the integration phases), the following table outlines the primary areas of ownership based on the progress report and repository structure.

Table 12.1: Team Contribution Matrix

Contributor	Main areas	Evidence Source	Status
Fernando T.H.L. (Harshana)	Sequencer architecture, JSON-RPC, scheduling policies	Commits, Progress Report	To be verified
Gamage M.S. (Malindu)	Submitter, L1 integration, DA formatting	Commits, Progress Report	To be verified
Lishan S.D.	Executor adaptation, gRPC, Prover integration	Commits, Progress Report	To be verified
Perera M.V.D.P.D.S. (Shalindra)	Workload Data Gen, Tools, Contracts, Orchestration	Commits, Progress Report	To be verified

Table 12.2: Contributor Verification Table

Contributor	Draft Role	Status
Harshana	Sequencer	To be verified
Malindu	Submitter	To be verified
Lishan	Executor	To be verified
Shalindra	Data Tools	To be verified

12.3.1 Harshana’s Contribution

Harshana focused on the Sequencer architecture, Domain-Driven Design integration, JSON-RPC API, scheduling policies, and core batching logic.

12.3.2 Malindu's Contribution

Malindu developed the Submitter, Layer-1 integration, and Data Availability formatting.

12.3.3 Lishan's Contribution

Lishan adapted the Executor, integrated gRPC communication, and managed Prover integration.

12.3.4 Shalindra's Contribution

Shalindra created the Workload Generator, Data Tools, Smart Contracts, and Pipeline Orchestration.

Chapter 13

Conclusion and Future Work

13.1 Conclusion

This report demonstrated the design, implementation, and evaluation of a modular ZK-Rollup benchmarking prototype, aimed at addressing the critical scalability challenges inherent in Ethereum Layer-1. Unlike production-grade rollups—which prioritize EVM equivalence and reliability at the cost of experimental flexibility—this prototype was explicitly architected to isolate and evaluate distinct performance bottlenecks under controlled, synthetic conditions.

By cleanly separating the sequencing, execution, proving, and submission stages, the framework allowed for a systematic analysis of the fundamental throughput–latency–cost trade-offs in Layer-2 scaling. The empirical observations yielded several key conclusions:

1. **The Batching Dilemma:** Larger batch sizes significantly reduce the amortized Layer-1 gas cost per transaction, but directly penalize early-arriving users by increasing queueing latency (delaying soft finality).
2. **Data Availability Dynamics:** Transitioning from standard transaction `calldata` to EIP-4844 blobs drastically reduces the L1 execution gas footprint. This structural efficiency gain allows rollups to operate more cost-effectively, fundamentally shifting the profitability curve of Layer-2 submission.
3. **Prover Bottlenecks:** Through parameterized mock proving, the research demonstrated that if proof generation is significantly delayed, the system becomes strictly compute-bound. This creates severe downstream backpressure, widening the gap between soft and hard finality regardless of batching efficiency.

Ultimately, the RollupX prototype successfully serves its purpose as a reproducible research framework, validating that scalability optimizations cannot be achieved in isolation. Enhancing one dimension (e.g., cost) inevitably impacts others (e.g., latency or computational overhead).

13.2 Future Work

While this project establishes a robust baseline for evaluating ZK-Rollup configurations, several avenues for future research remain:

- **Real Prover Integration:** Transitioning from the benchmark-mode mock prover to a fully integrated, hardware-accelerated cryptographic prover cluster (e.g., utilizing advanced recursive PLONK or STARK circuits) to measure absolute proof generation costs under heavy, parallelized load.
- **Trace-Driven Workloads:** Replacing the synthetic Poisson-based workload generator with trace-driven inputs based on historical mainnet dataset replays (such as actual zkSync Era or Arbitrum traces) to provide greater external validity.
- **Dynamic Batching Policies:** Implementing and formally evaluating dynamic, adaptive batching algorithms that automatically adjust size and timeout triggers in real-time based on fluctuating Layer-1 base fees and mempool congestion.
- **Off-Chain DA Layers:** Expanding the Data Availability scope beyond L1 call-data/blobs by fully integrating with external DA networks (such as Celestia or EigenDA) to benchmark the trade-offs of Validium or Volition architectures.

Future work could include replacing synthetic workloads with trace-driven workloads utilizing the public ZKsync dataset [?], incorporating realistic DeFi workloads [?], and performing real prover benchmarking on accelerated hardware.

Bibliography

- [1] Eli Ben-Sasson et al. Scalable, transparent, and post-quantum secure computational integrity, 2018.
- [2] Vitalik Buterin et al. A next-generation smart contract and decentralized application platform. *white paper*, 3(37):2–1, 2014.
- [3] Chainspect. Solana vs ethereum tps comparison. <https://chainspect.app/compare/solana-vs-ethereum>, 2025. Accessed: 2026-05-10.
- [4] S. Chaliasos et al. Benchmarking layer-2 systems, 2024.
- [5] Stefanos Chaliasos, Itamar Reif, Adrià Torralba-Agell, Jens Ernstberger, Assimakis Kattis, et al. Analyzing and benchmarking zk-rollups. In *6th Conference on Advances in Financial Technologies (AFT 2024)*, volume 316 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 6:1–6:23, 2024.
- [6] Ethereum. Eip-4844: Shard blob transactions, 2022.
- [7] Ethereum Foundation. Gas and fees. <https://ethereum.org/developers/docs/gas/>, 2025. Accessed: 2026-05-10.
- [8] Ethereum Foundation. Introduction to smart contracts. <https://ethereum.org/developers/docs/smart-contracts/>, 2025. Accessed: 2026-05-10.
- [9] Ethereum Foundation. Layer 2. <https://ethereum.org/layer-2/learn/>, 2025. Accessed: 2026-05-10.
- [10] Ethereum Foundation. Proof-of-stake (pos). <https://ethereum.org/developers/docs/consensus-mechanisms/pos/>, 2025. Accessed: 2026-05-10.
- [11] Ethereum Foundation. Scaling. <https://ethereum.org/developers/docs/scaling/>, 2025. Accessed: 2026-05-10.
- [12] Etherscan. Ethereum (eth) blockchain explorer. <https://etherscan.io/>, 2025. Accessed: 2026-05-10.
- [13] Y. Fu et al. The blockchain trilemma, 2024.
- [14] A. Gogol et al. Scaling defi with zk rollups. *arXiv preprint*, 2025.
- [15] Jens Groth. On the size of pairing-based non-interactive arguments, 2016.
- [16] A. Habib et al. Bottlenecks in zk-rollups, 2025.

- [17] L. Lavour et al. Modular blockchains, 2023.
- [18] S. Motepalli et al. Sequencer policies in rollups, 2023.
- [19] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf>, 2008.
- [20] A. Saif et al. Data availability in rollups, 2024.
- [21] Abhilash I. Sanka and Ray C.C. Cheung. A systematic review of blockchain scalability: Issues, solutions, analysis and future research. *Journal of Network and Computer Applications*, 195:103232, 2021.
- [22] N. Silva et al. zksync era public dataset, 2025.
- [23] Anh C Tran, Vu V Thanh, Nguyen C Tran, and Hien T Nguyen. An implementation and evaluation of layer 2 for ethereum with zk-rollup. In *Lecture Notes in Computer Science*, pages 57–63. Springer, 2023.
- [24] Gavin Wood et al. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 151(2014):1–32, 2014.
- [25] Qiheng Zhou, Huawei Huang, Zibin Zheng, and Jing Bian. Solutions to scalability of blockchain: A survey. *IEEE Access*, 8:16440–16455, 2020.

Appendix A

Repository Structure

An overview of the essential directories within the RollupX repository:

```
.
|-- benchmark-suite/    # Workload generator and orchestration scripts
|-- contracts/          # L1 Smart Contracts (Solidity/Hardhat)
|-- data-tools/         # Python scripts for telemetry aggregation and plotting
|-- docs/               # System architecture and component documentation
|-- executor/           # Rust executor (state transitions and proving integration)
|-- risc0_prover/       # Experimental RISC0 zkVM prover backend
|-- sequencer/          # Rust sequencer (transaction ingestion and batching)
|-- submitter/          # Rust submitter (DA formatting and L1 interaction)
|-- compose/            # Docker compose configuration files
|-- latex/              # LaTeX sources for reports, papers, and proposals
```

Appendix B

Build and Run Guide

This appendix outlines the steps required to build and run the RollupX prototype natively for development purposes (bypassing Docker Compose).

Prerequisites:

- Node.js (v18+) and npm
- Rust (stable and nightly-2024-08-01 toolchains)
- Python 3.11+
- SQLite3 CLI, nasm, and protobuf-compiler

Build Commands:

```
# Install contract dependencies
cd contracts && npm install
npx hardhat compile

# Build core Rust components
cd risc0_prover && cargo +stable build -p rollup_host && cd ..
cd executor && cargo build --ignore-rust-version && cd ..
cd sequencer && cargo build --release && cd ..
cargo build --manifest-path submitter/Cargo.toml
```

Run Commands (in separate terminals):

```
# Terminal A: Start L1 Node
cd contracts && npx hardhat node

# Terminal B: Deploy Contracts
cd contracts && npx hardhat run scripts/deploy-local.ts --network localhost

# Terminal C: Start Executor
export PROVER_BACKEND=risc0 ... # (Set environment variables)
./executor/target/debug/zksync_state_machine

# Terminal D: Start Submitter
cargo run --manifest-path submitter/Cargo.toml --bin submitter -- \
```

```
--config submitter/config/local-offchain-risc0.yaml  
  
# Terminal E: Start Sequencer  
cd sequencer && RUST_LOG=info cargo run --release
```

Appendix C

Benchmark Execution Guide

Running an Automated Benchmark (Dockerized):

```
# Start the core infrastructure
docker compose --profile core up -d

# Execute the specific experiment matrix
bash benchmark-suite/scripts/run_matrix.sh --phase feasibility
```

Collecting and Plotting Metrics:

```
# Trigger the data tools analytics pipeline
bash run_pipeline.sh
```

Appendix D

Configuration Reference

The system behavior is largely controlled by TOML and YAML files. Table D.1 highlights key parameters.

Table D.1: Key Configuration Parameters

Parameter	Location	Default	Effect
<code>max_batch_size</code>	<code>sequencer.toml</code>	100	Triggers a batch seal when the tx pool reaches this size.
<code>timeout_interval_ms</code>	<code>sequencer.toml</code>	5000	Triggers a batch seal if the interval expires, regardless of size.
<code>policy_type</code>	<code>sequencer.toml</code>	FCFS	Determines the transaction ordering algorithm (e.g., FCFS or FeePriority).
<code>da_mode</code>	<code>submitter.yaml</code>	<code>calldata</code>	Swaps the L1 data posting mechanism between standard <code>calldata</code> and EIP-4844 blobs.

Appendix E

Metrics Output Schema

The Workload Generator outputs a JSON file (`workload_<exp_id>.json`) capturing the global experiment parameters and a CSV file (`tx_log_<run_id>.csv`) tracking individual transactions.

tx_log schema:

- `tx_hash`: Unique transaction identifier.
- `submit_time`: Timestamp (ms) when the workload generator sent the request.
- `status`: Success or Failure.
- `latency_ms`: The Round-Trip Time observed by the client (approximating soft finality).

The Submitter outputs `submitter_metrics.json`, containing:

- `batch_id`: Internal ID.
- `l1_tx_hash`: The Ethereum transaction hash.
- `gas_used`: The exact L1 gas consumed by the verification transaction.
- `da_mode`: The DA formatting used (Calldata/Blob).
- `total_cost_eth`: The calculated economic cost of the submission.

Appendix F

Important Algorithms

This appendix provides simplified pseudocode for the Sequencer's core batch orchestration logic, demonstrating the interaction between the size trigger and the timeout trigger.

Algorithm: Batch Orchestrator Loop

Input: `max_batch_size`, `timeout_interval`

State: `current_batch_txs = []`, `last_seal_time = now()`

loop indefinitely:

 // Pull transactions from the pool ordered by the active policy

`tx = tx_pool.pop()`

 if tx is not null:

`current_batch_txs.append(tx)`

`time_elapsed = now() - last_seal_time`

 if `current_batch_txs.length >= max_batch_size` OR `time_elapsed >= timeout_interval`

 if `current_batch_txs.length > 0`:

`seal_and_dispatch(current_batch_txs)`

`current_batch_txs = []`

`last_seal_time = now()`

`sleep(tick_duration)`